

Report for assignment 4 - Group 16

Project

Name: sir-lancebot

URL: <https://github.com/python-discord/sir-lancebot>

A Discord bot started as a community project for Hacktoberfest 2018 and later evolved into an introductory project for aspiring developers starting out in open source development.

Onboarding experience

Did you choose a new project or continue on the previous one? If you changed the project, how did your experience differ from before?

We chose a new project for this assignment. Onboarding was more involved this time than last because we had to set up Python, Docker, a Discord test server, and a test bot. The issue originated in the `python-discord/bot` repository, but halfway through our assignment, the maintainers decided to move the issue to a sister repository, `sir-lancebot`, so we had to redo our onboarding. Luckily, the process was essentially the same, but we could not run any existing tests because the new repository had none.

Effort spent

Oskar:

1. ~ 3 h of plenary discussions and issue searching.
2. ~ 7 h discussing with the group.
3. ~ 30 min reading documentation
4. ~ 30 min setting up Python env, Docker, Discord, and the bot
5. ~ 15 min analyzing code (this is done continuously throughout the assignment, so hard to measure realistically)
6. ~ 1:30 h writing documentation
7. ~ 15 h writing code
8. 10 min running code

Markus:

1. Around 4h
2. Some amount then :P (if we count Discord messages?)
3. Around 10 min for every implementation
4. About 1h:
 - a. Python 3.14
 - b. Docker

- c. Setting up the bot on discord developer site
 - d. Get the bot running
- 5. Maybe 5-10 min for every implementation
- 6. 1h so far + creating the issues
- 7. 12.0 h (lost track)
- 8. 10-30 min running code

Elin:

- 1. 3h for meetings and choosing issue
- 2. 3h discussions in meetings and through text
- 3. 2h
- 4. 1.5h managing python versions and troubleshooting bot startup
- 5. 2h mainly for understanding unit testing in python
- 6. 4h
- 7. 2h
- 8. 0.5h

Ben:

- 1. 3h
- 2. 4h
- 3. 1 hr 30 min reading documentation
- 4. 3 hrs 30 min onboarding
- 5. 2h understanding code structure
- 6. 5 hrs 15 min working on UML diagram, essence
- 7. 2 hrs 30 min writing code
- 8. 30 min

Ali:

- 1. 3h30
- 2. 5h
- 3. 2 hours (Most of it looking at GITHUB API and discord bot)
- 4. 2h30:
 - Python
 - Docker (2x I had a problem and had to reinstall it)
 - uv
 - .env
 - Discord Bot
- 5. 4 hours (lot of debugging especially for an issue where github failed to a behaviour of API endpoint)
- 6. 30 minutes of writing documentation
- 7. 9 hours
- 8. 30 minutes running code

Overview of issue(s) and work done.

Title: Feature: add a !stats command for stats on Python discord repos #1724

URL: <https://github.com/python-discord/sir-lancebot/issues/1724>

The requested feature is a command for the Discord bot "sir-lancebot" that outputs GitHub statistics from a GitHub repo into a Discord channel. These statistics include, for example, the number of commits within a certain time period.

Scope (functionality and code affected).

The bot follows a modular architecture in which each command is an independent module. Commands are decoupled; they operate within their own scope, preventing side effects on the rest of the codebase. Because a `githubinfo` command already existed, the maintainers instructed us to add our feature as a subcommand to the existing command. This meant modifying the `githubinfo.py` file without changing the functionality of existing code. The only functionality we changed was a bug we discovered and fixed while adding the code. This is noted in a bug report issue: <https://github.com/python-discord/sir-lancebot/issues/1728>

Requirements for the new feature or requirements affected by functionality being refactored

Since the command is called with `!gh stats <start_date> <end_date> <repo>`, the main requirement was that the input was formatted correctly. Other focuses included that the bot actually printed the correct output, formatted as requested. See Table 1 below:

Since our implementation requires frequent calls to the GitHub API, we need to generate a GitHub token at <https://github.com/settings/tokens> (A classic one with access to a private repo should suffice).

After adding it to the environment, the `.env` should look something like this:

```
CLIENT_TOKEN=  
CLIENT_GUILD=  
ROLES_ADMIN=  
ROLES_HELPERS=  
CHANNELS_ANNOUNCEMENTS=  
CHANNELS_DEVLOG=  
CHANNELS_SIR_LANCEBOT_PLAYGROUND=  
TOKENS_GITHUB=
```

Optional (point 3): trace tests to requirements.

Table 1: Requirement trace

Requirement ID	Requirement Title	Associated Unit Test	Description of Verification
REQ-01	Date Format Validation	test_validate_date_accepts_valid_formats	Verifies the regex safely accepts correctly formatted dates and rejects bad strings.
REQ-02	Date Range Logic	test_validate_date_range_rejects_wrong_order	Verifies the bot correctly rejects inputs where the start date is after the end date.
REQ-02	Date Range Logic	test_validate_date_range_accepts_same_day	Verifies the bot allows querying a single day.
REQ-03	Aggregate Issue Statistics	test_get_issue_count_success	Verifies the method correctly extracts the total_count integer from the mocked GitHub JSON response.
REQ-04	Aggregate Pull Request Statistics	test_get_pr_count_merged	Verifies the method correctly parses PR-specific data states.

REQ-05	Aggregate Commit Statistics	test_get_commit_count_multiple_pages	Verifies the regex correctly extracts the last page number from a mocked Link header.
REQ-06	Graceful API Error Handling	test_api_failure_returns_negative_one	Verifies the bot safely returns -1 when a mocked API responds with a 404 error code.
REQ-07	Inline Codeblock Parsing	test_remove_codeblocks_inline	Verifies the updated regex successfully strips backticks from the middle of a sentence.

Code changes

Patch

The implemented changes for the patch can be seen here:

<https://github.com/python-discord/sir-lancebot/pull/1729>

Optional (point 4): the patch is clean.

NOTE: The tests are presented in another branch per the maintainers' instructions; they did not want us to include tests with this patch. Because we got good last-minute feedback from the maintainer that would change our tests, we decided not to submit the PR yet. We will present our tests as they were to the TA on a remote branch.

Optional (point 5): considered for acceptance (passes all automated checks).

The PR passed all automatic checks, but we still need to make changes based on the feedback we received and resubmit.

Test results

Before:

N/A: There were no tests.

After:

```
$ uv run task test
```

```
2026-03-01 22:58:02 | INFO | root | Logging initialization complete
```

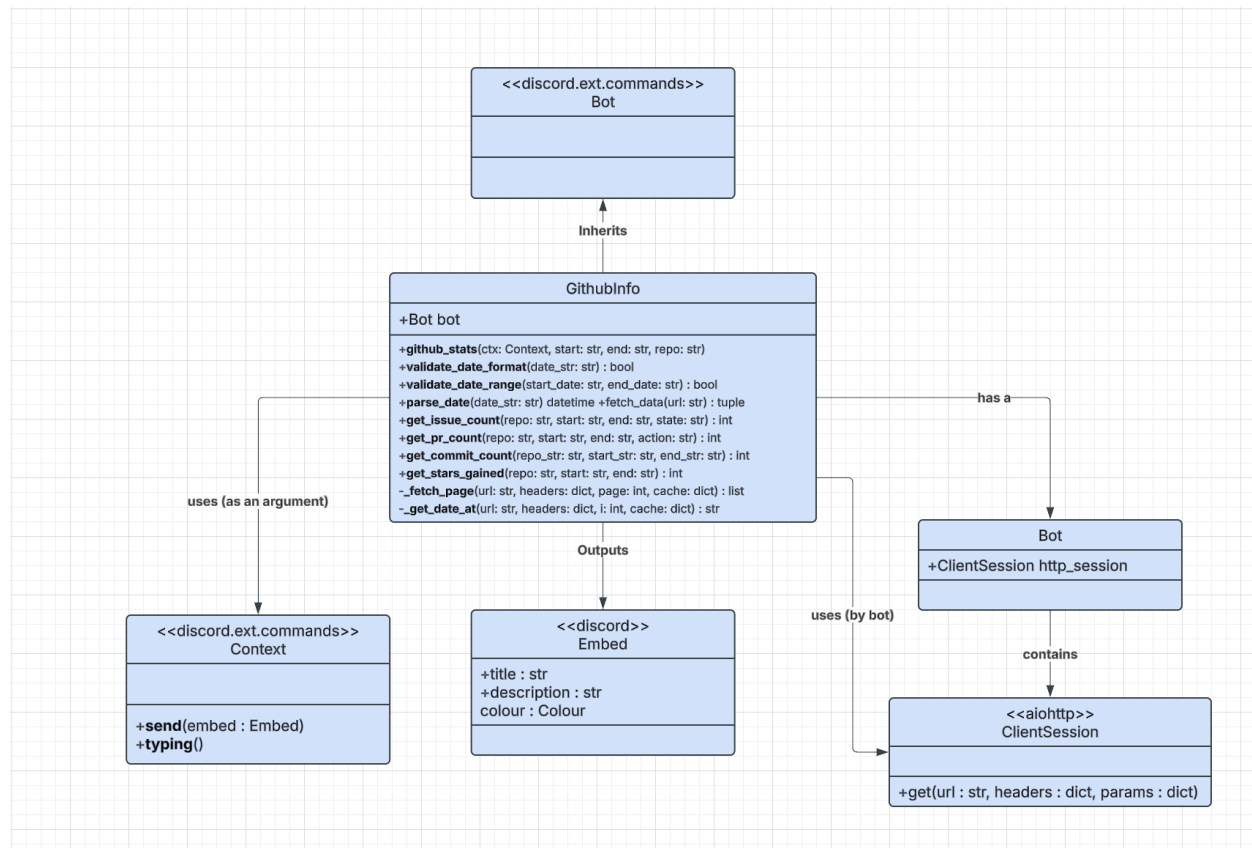
```
2026-03-01 22:58:02 | DEBUG | pydis_core.utils._monkey_patches | Patching send_typing, which should fix things breaking when Discord disables typing events. Stay safe!
```

```
.....
```

```
-----  
Ran 8 tests in 0.019s
```

```
OK
```

UML class diagram and its description



The UML class diagram illustrates the modular design of the feature within the sir-lancebot architecture.

- **GithubInfo**: This is the central class we modified. It is a Cog (a module in the discord.py framework) that encapsulates all GitHub-related state and behavior.

- **Bot:** Through the Bot, Discord accesses an asynchronous `aiohttp.ClientSession` to make non-blocking HTTP requests to the GitHub API.
- **Context:** The main entry point for our feature (`github_stats`) takes a Context object as an argument. This object holds the state of the user's message and provides the `send()` method to reply.
- **Embed:** The output of our command is a `discord.Embed` object, which structures the JSON payload sent back to Discord to display the statistics in a UI.

Key changes/classes affected

The scope of our issue was highly localized to prevent unintended side effects on the rest of the bot. The only class that was structurally modified was `Githubinfo` (`bot/exts/utilities/githubinfo.py`).

Within this class, we introduced several new operational methods:

1. **Validation Methods:** ``parse_date``, ``validate_date_format``, and ``validate_date_range`` were added to sanitize user input before making any network requests.
2. **API Fetchers:** ``get_issue_count``, ``get_pr_count``, ``get_commit_count``, and ``get_stars_gained`` were added to encapsulate the specific query logic for different GitHub API endpoints.
3. **Command Route:** The ``@github_group.command(name="stats")`` was added to expose the ``github_stats`` asynchronous function to end-users.
4. **Refactored Regex:** The existing ``remove_codeblocks`` static method was refactored by removing a ``^`` anchor to correctly parse inline code blocks regardless of their position in a string.

Optional (point 1): Architectural overview

The bot is an event-driven, asynchronous Python application built on top of the `discord.py` API wrapper.

The architecture is strictly modular and component-based using the `discord.ext.commands.Cog` framework. Each "Cog" represents a feature.

Because Discord is an environment where thousands of users might trigger commands simultaneously, the architecture relies heavily on Python's `asyncio` library. All network I/O is delegated to `aiohttp`. This asynchronous architecture ensures that while our command is waiting for GitHub to return pagination data, the bot's main event loop is never blocked, allowing it to continue responding to other users seamlessly.

Optional (point 2): relation to design pattern(s).

In this project, we mainly have used design patterns such as the **Command Pattern**, the **Facade Pattern**, and **Dependency Injection**. Most notable being the Facade pattern, where the ``GithubInfo`` methods act as a Facade for the GitHub REST API, hiding all complexity behind a single command. The user only needs to provide a date range and a repository name,

and our Facade handles the rate-limit checks, the $O(\log N)$ binary-search caching, and the asynchronous I/O behind the scenes.

Overall experience

What are your main takeaways from this project? What did you learn? How did you grow as a team, using the Essence standard to evaluate yourself?

Our main takeaways can be summarized into two points.

1. The development of open source projects does not work the same way as how we have worked in the course so far. When looking for an issue for this assignment, we saw that some issues had a lot of work done on them, but still lacked an assigned person. Additionally, issues can seem to have been worked on before, as indicated by a PR added to them, but the PR may be stale, and someone has forgotten to close it. In contrast, we have always assigned individuals to issues to ensure it is clear what everyone needs to do. Therefore, we have learned that open source projects can look different and that formality can be optional.
2. One of the most confusing aspects of our assignment was that the core maintainers were discussing how to implement our chosen issue while we were working on it. They, for example, decided that it would be best to move the issue to another bot and discussed what functionality should actually be included. This meant our tasks could change depending on the maintainers' decisions. From this, we learned that open source projects are alive and can change in response to many factors. This is perhaps the biggest challenge when comparing school projects with real life: in open source, working on a project for an employer, requirements can change midway through, and you have to be able to pivot. In this sense, this was a great learning opportunity.

ESSENCE

Table 2: Essence

State	Checklist
Seeded	☒ The team mission has been defined in terms of the opportunities and outcomes. ☒ Constraints on the team's operation are known. ☒ Mechanisms to grow the team are in place. ☒ The composition of the team is defined. ☒ Any constraints on where and how the

	work is carried out are defined. ☒ The team's responsibilities are outlined. ☒ The level of team commitment is clear. ☒ Required competencies are identified. ☒ The team size is determined. ☒ Governance rules are defined. ☒ Leadership model is determined.
Formed	☒ Individual responsibilities are understood. ☒ Enough team members have been recruited to enable the work to progress. ☒ Every team member understands how the team is organized and what their individual role is. ☒ All team members understand how to perform their work. ☒ The team members have met (perhaps virtually) and are beginning to get to know each other. ☒ The team members understand their responsibilities and how they align with their competencies. ☒ Team members are accepting work. ☒ Any external collaborators (organizations, teams and individuals) are identified. ☒ Team communication mechanisms have been defined. ☒ Each team member commits to working on the team as defined.
Collaborating	☒ The team is working as one cohesive unit. ☒ Communication within the team is open and honest. ☒ The team is focused on achieving the team mission. ☒ The team members know and trust each other.
Performing	☒ The team consistently meets its commitments.

	☒ The team continuously adapts to the changing context. ☐ The team identifies and addresses problems without outside help. ☐ Effective progress is being achieved with minimal avoidable backtracking and reworking. ☐ Wasted work and the potential for wasted work are continuously identified and eliminated.
Adjourned	☐ The team responsibilities have been handed over or fulfilled. ☐ The team members are available for assignment to other teams. ☐ No further effort is being put in by the team to complete the mission

In what state are you in? Why? What are obstacles to reach the next state?
How have you improved during the course, and where is more improvement possible?

We are still in the Performing state, as we have been for most assignments throughout this course. When it comes to this assignment, we have not been entirely able to fulfill the checkpoint regarding "The team identifies and addresses problems without outside help," as throughout this project, we have been communicating requirements with the creators of the feature we are trying to implement.

To reach the next state, we would need to be quite proficient in the area we are implementing, as this is a new feature for most of us.

Throughout the course, we have always reached the performing state but never reached the adjourned state. This could be due to a variety of reasons, for instance, the projects being quite different from each other every time, which requires us to redefine our responsibilities within the team and how we work, whereas sticking to one main area might have allowed us to become relatively experienced in a specific field. Perhaps, if we look at using GitHub, we could reach the adjourned, as it is something we have used constantly throughout this course. However, it is difficult to identify an area where further improvement is needed beyond allowing us more time to deepen our learning and skills, which we unfortunately don't have.

Optional (point 6): How would you put your work in context with best software engineering practice?

During our development, we have generally used the workflow of SEMAT without using the cards intentionally. The alphas “Opportunity” and “Stakeholders” were present through the actual issue and the issue owner who wanted to see this feature implemented. In the beginning, the stakeholders were not very involved as we mainly asked if it was okay for our group to take on this issue. As we progressed, however, we had more contact with them via GitHub and it led to us being able to refine the requirements further. In hindsight, we should have included and asked more questions from the very beginning, like the “Stakeholders” card suggests with the different levels of the card. We have yet to achieve the last levels, however, since our PR has not been accepted yet.

In regards to the actual development, we utilized test driven development and let the defined requirements define how the code. This can be seen in the “Requirements” card which states that the software system should satisfy the stakeholders wants. We have essentially gone through all the steps in the card by first defining requirements, getting feedback from stakeholders, rewriting the requirements, implementing tests and finally writing the actual implementation. After the requirements were done, we were able to start working with “Software System” where we defined how our functionality should fit in with the rest of the bot architecture.

One challenging part of our project was the constant input of stakeholders and the migration of our issue to another bot in the middle of our project. This made the levels “Started” and “Under control” in the “Work” alpha hard to achieve since we essentially had to restart it halfway through. This tested our agility as we quickly needed to do another onboarding and then make sure that the team members could get back to working. After that change, we believe that we got better at managing unexpected problems and refactorings. For example, we realized that the project did not want us to include our test in our submitted patch. This led to us having to restructure the default branches in our forks by removing all tests from the branch we had been working on.

Lastly, a famous rule in software engineering is the [“Boy Scout Rule”](#) which means that you should “Leave your code better than you found it”. This is generally good practice and we had the opportunity to do this by fixing a pre-existing bug with our patch.

Optional (point 7): Is there something special you want to mention here?

For ``get_stars_gained``, we implemented an $O(\log N)$ binary search with a request cache to navigate a paginated REST API and bypass rate limits.